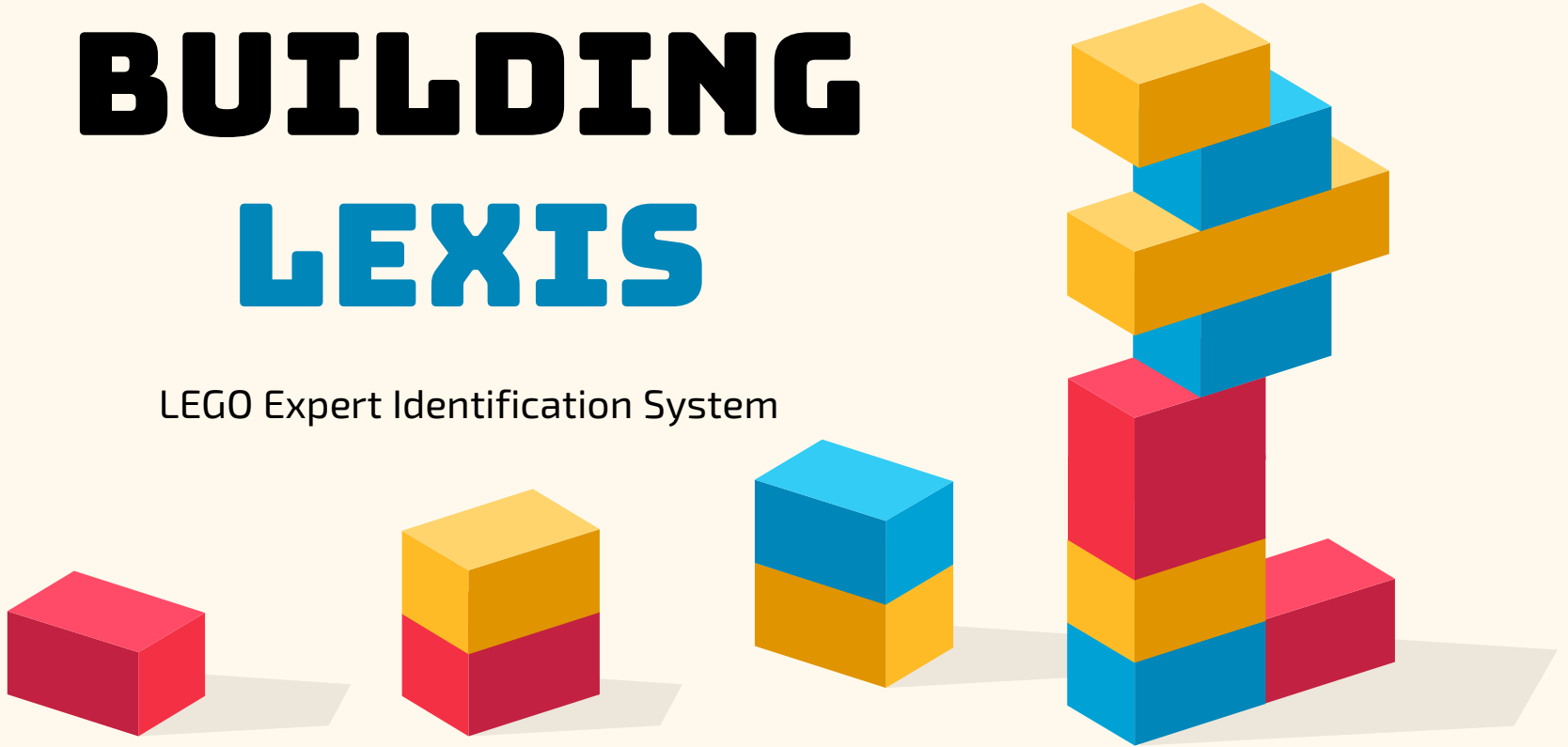


BUILDING LEXIS

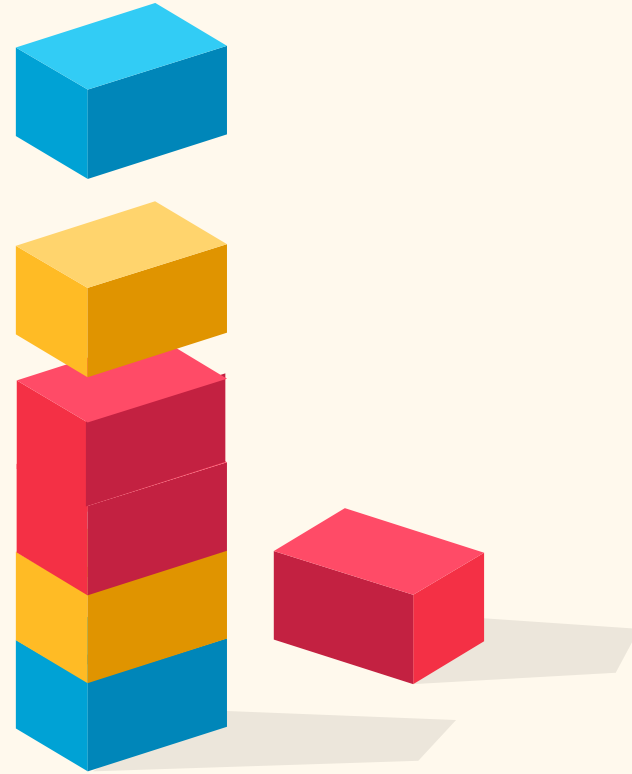
LEGO Expert Identification System



Done by : Yasmin Alhendawy, Mohammed Eltallaa

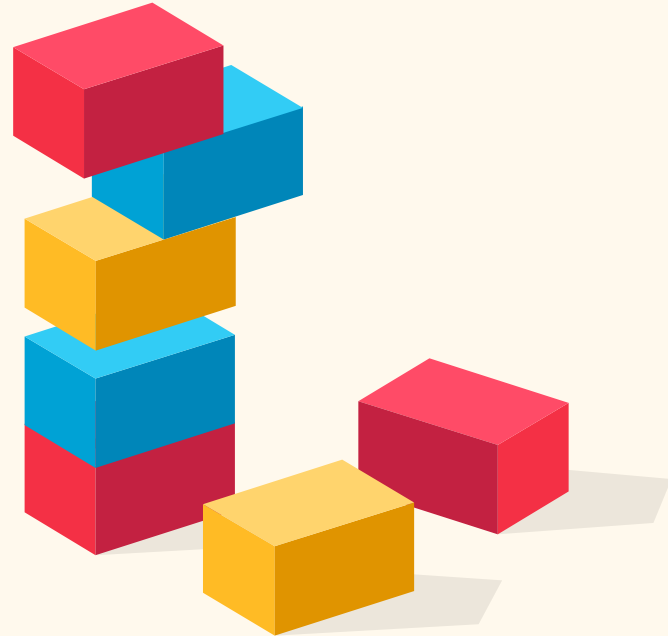
TABLE OF CONTENTS

- 01** Introduction
- 02** Model Training
- 03** Workflow Architecture
- 04** Results
- 05** Conclusion

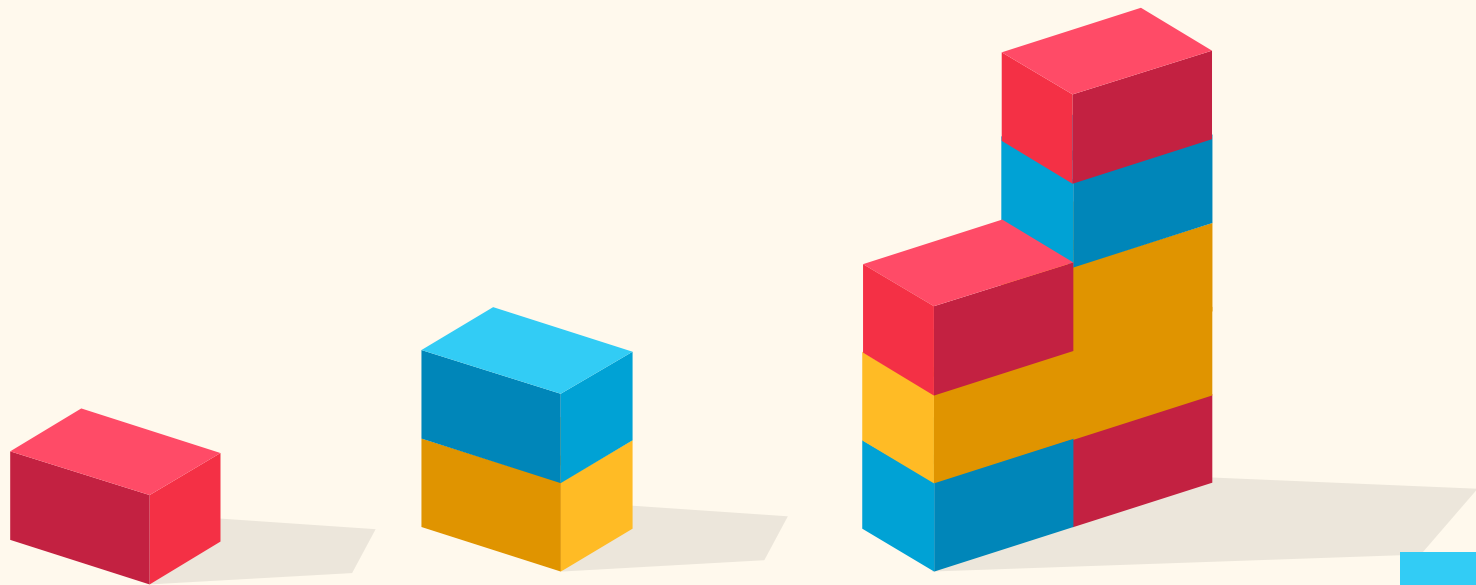


WHAT LEXIS DOES ?

- LEXIS is a Real-time AI-powered system which:
 - Detects, classifies, and colour-matches LEGO pieces
 - Combines YOLOv11n object detection with MobileNetV2-based CNN classifiers
 - Works with live webcam, phone camera, or image folders
 - Outputs results to Excel for tracking & analysis



MODELS TRAINING



MODELS



CNN23

Trained on 23 classes
for full classification



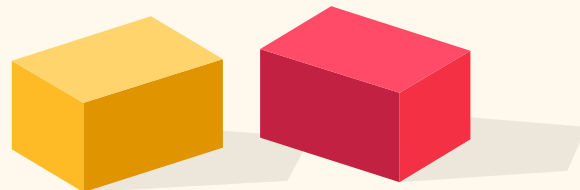
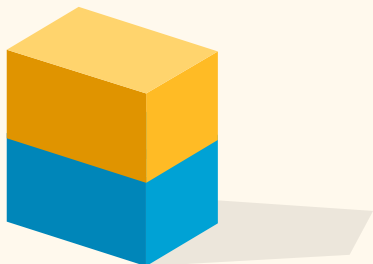
CNN7

Lightweight version
used for semi stacking



YOLO

Trained on 14 classes
for object detection



46,000

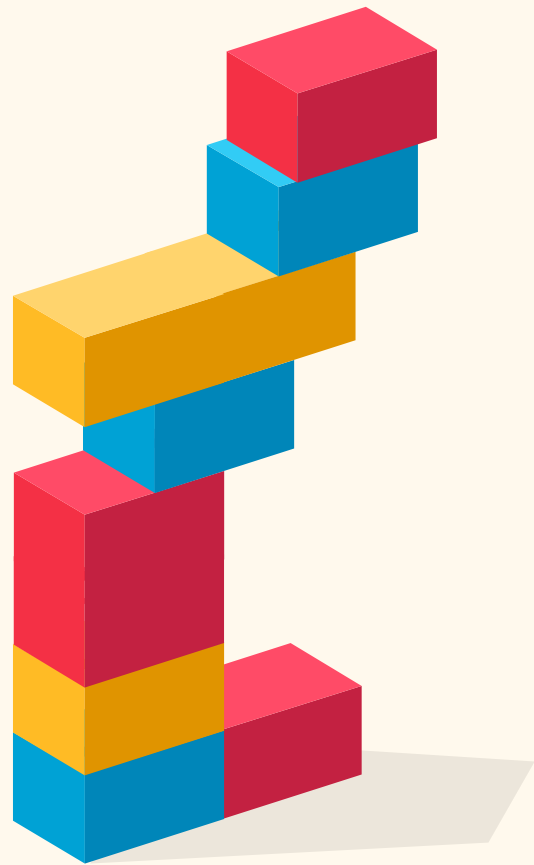
hand-labelled real life LEGO part images across 23 classes.

5,200

synthetic 3rd generated data of LEGO parts for 7 classes.

1,100

yolo LEGO labelled images.





CNN23 & CNN7



1. Data Preparation

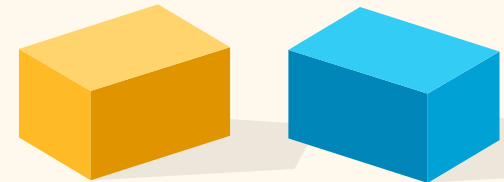
Up sampling data to Input size: 96×96

Augmentations:

Rotation, Zoom, Shifts

Shear, Brightness, Horizontal Flips

Split: 80% training, 20% validation





CNN23 & CNN7



2. Transfer Learning Setup

Base Model: MobileNetV2 (ImageNet weights)

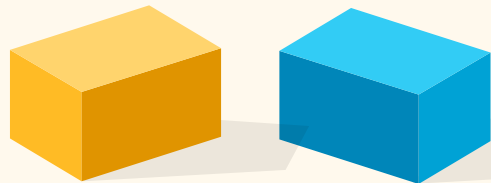
Head removed → Custom classifier added

Custom Head:

Global Average Pooling

Dense(128) → Dropout(0.5) → Softmax output

Result: A hybrid model combining pretrained power with task-specific learning.





CNN23 & CNN7



3. Training Configuration

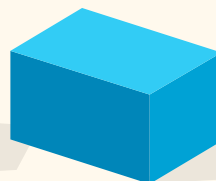
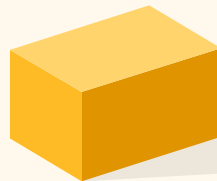
Optimizer: Adam with learning rate $1e-5$

Loss: Categorical Crossentropy

Metrics: Accuracy

Epochs: 50

Batch Size: 32





CNN23 & CNN7



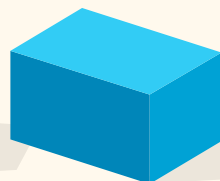
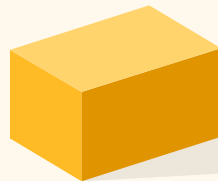
4. Smart Regularization

EarlyStopping: Halts training if no progress in 7 epochs

ReduceLROnPlateau: Cuts LR by 50% after 3 stagnant epochs

Class Weights: Adjusts for class imbalance

Ensures stable training, prevents overfitting, and boosts rare class performance





CNN23 & CNN7



5. Semi-Stacking Ensemble

To increase classification robustness:

Two models trained:

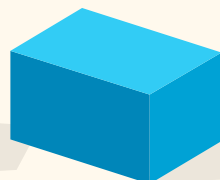
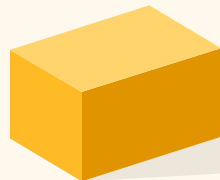
CNN7: Classifies 7 large/high-confidence parts

CNN23: Full classifier across all classes

Workflow Logic:

If part belongs to **ENSEMBLE_CLASSES**, blend predictions from both CNN7 and CNN23 using weighted averaging

This "semi-stacking" increases confidence and accuracy on key parts without full model redundancy





Model: YOLOv11n – fast, lightweight, optimized for real-time

The model is trained to detect very small Lego pieces

Training Setup:

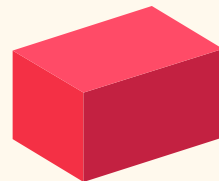
Classes: 14 selected LEGO types

Input Size: 640×640

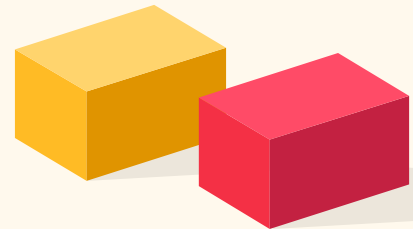
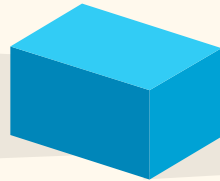
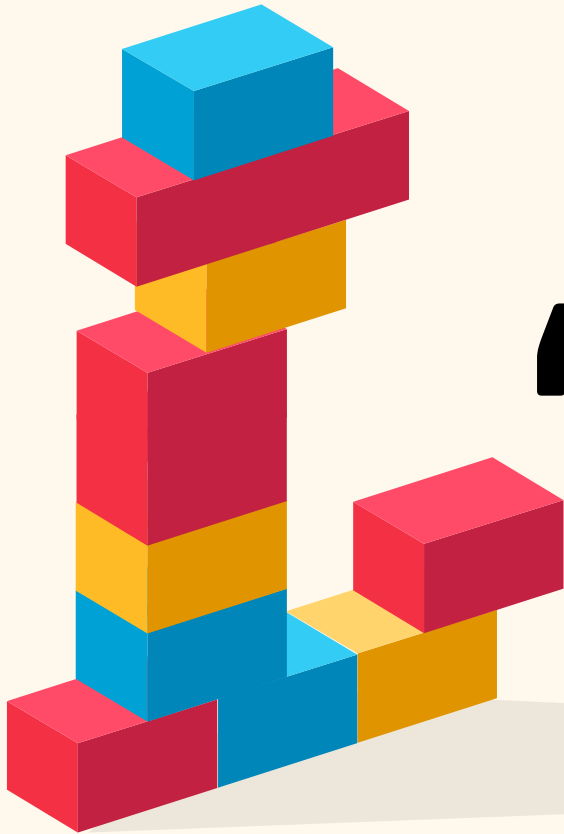
Epochs: 50 | **Batch Size:** 16

Augmentations: Flip, Rotate, Brightness

Framework: Ultralytics YOLO

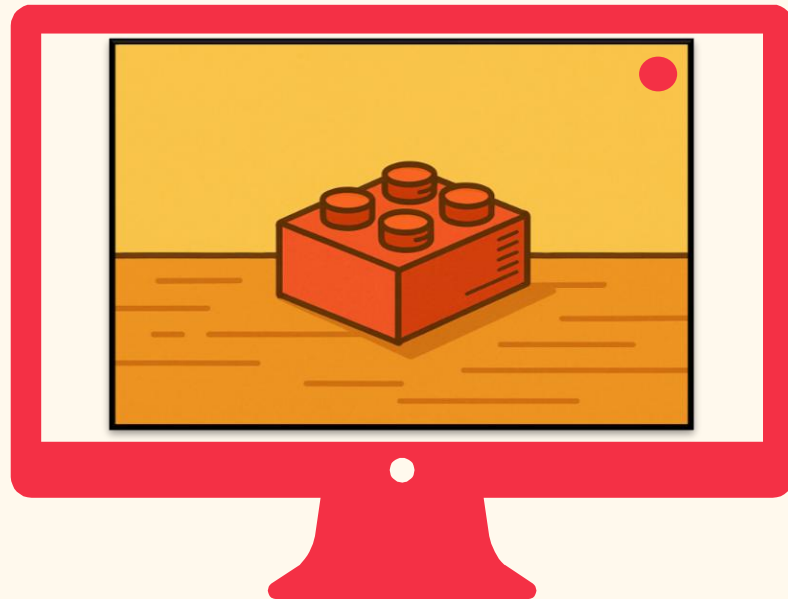


WORKFLOW ARCHITECTURE



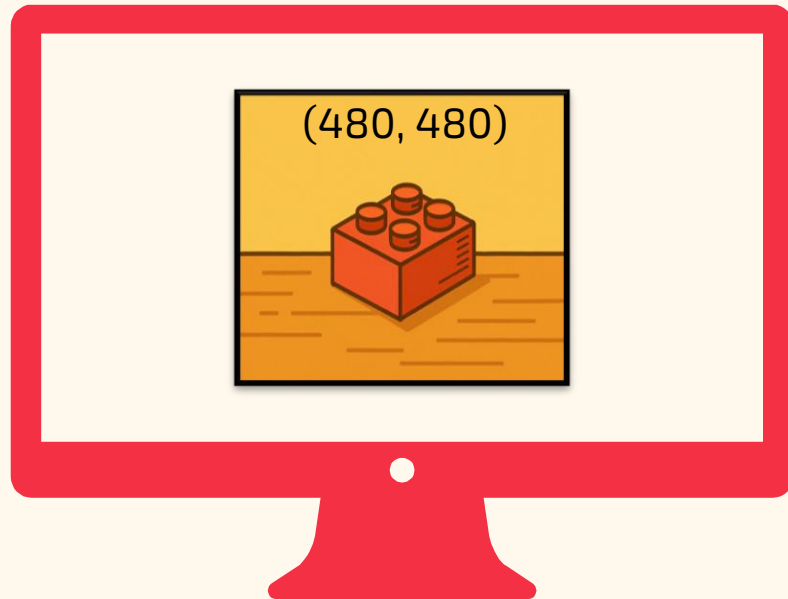
1. FRAME CAPTURE

- Frames are captured in real-time from either:
 - A webcam.
 - A phone camera streaming over local network.
- Each frame is analysed independently.



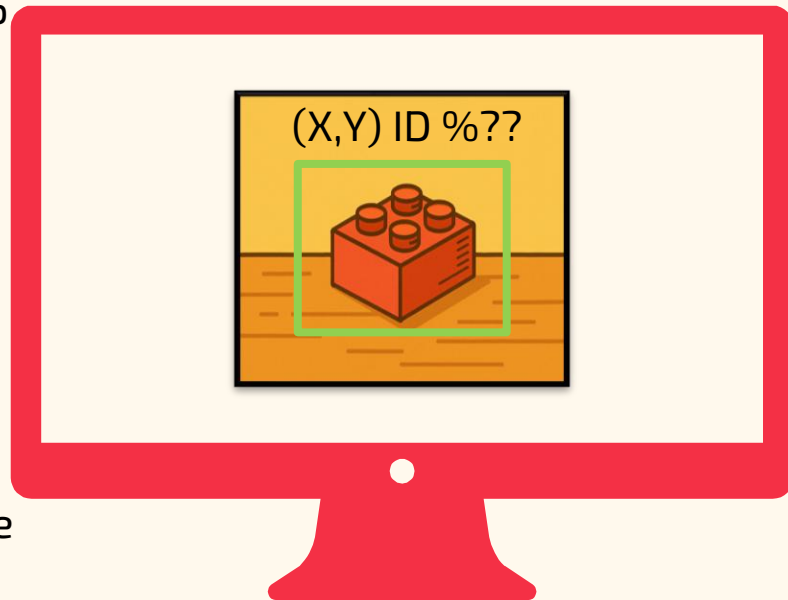
2. FRAME PREPARATION

- Frames are resized (e.g., (480, 480)) for uniform input
- Two copies are created:
 - One for display
 - One for analysis



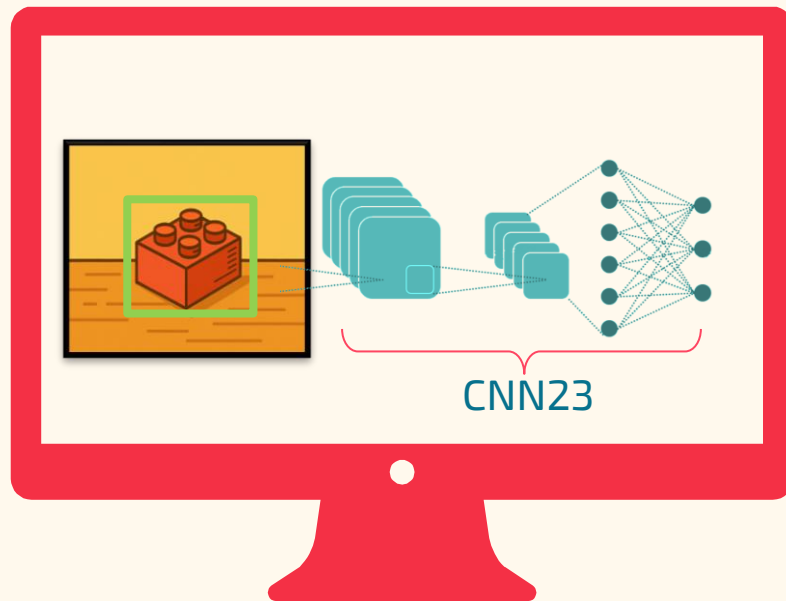
3. OBJECT DETECTION WITH YOLO

- YOLO runs on every n-th frame (e.g., every 6 frames)
- Output includes:
 - Bounding box coordinates
- Finds bounding boxes for LEGO parts
- Filters based on confidence and realistic size



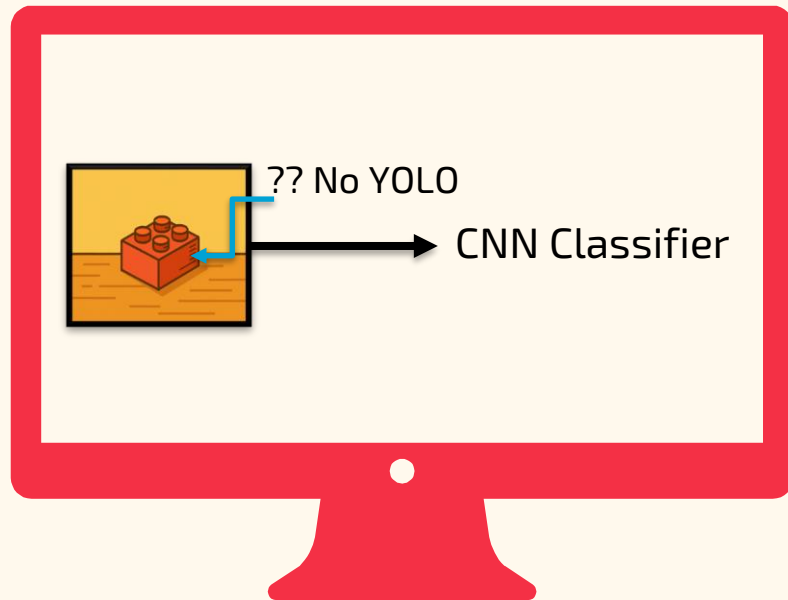
4. PATCH CLASSIFICATION

- Detections are filtered by:
 - Confidence threshold (to reduce noise)
 - Minimum and maximum bounding box area (to ignore irrelevant objects)
 - Extract patch from each detected box
 - Use MobileNetV2 (TFLite, 23 classes)
- If the class is sensitive → refine with second CNN (7-class model)
- If YOLO fails → fallback to centre crop classification



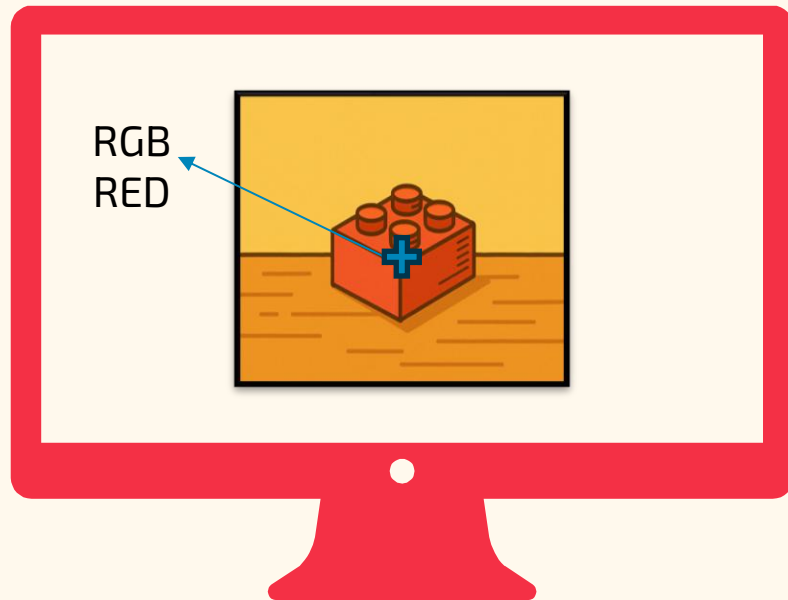
5. FALLBACK DETECTION

- When YOLO returns no detections:
 - A fallback crop from the frame centre is passed to the classifier
 - Ensures system continues functioning on difficult parts or bad lighting



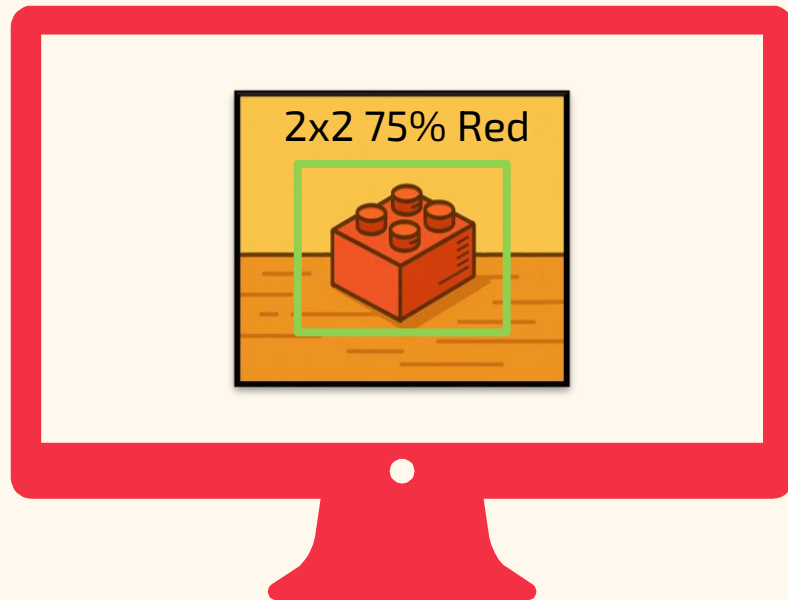
6. COLOUR DETECTION

- Centre pixel colour is extracted from patch
- Compared against LEGO colour database
- Best match (by BGR distance) is selected and labelled



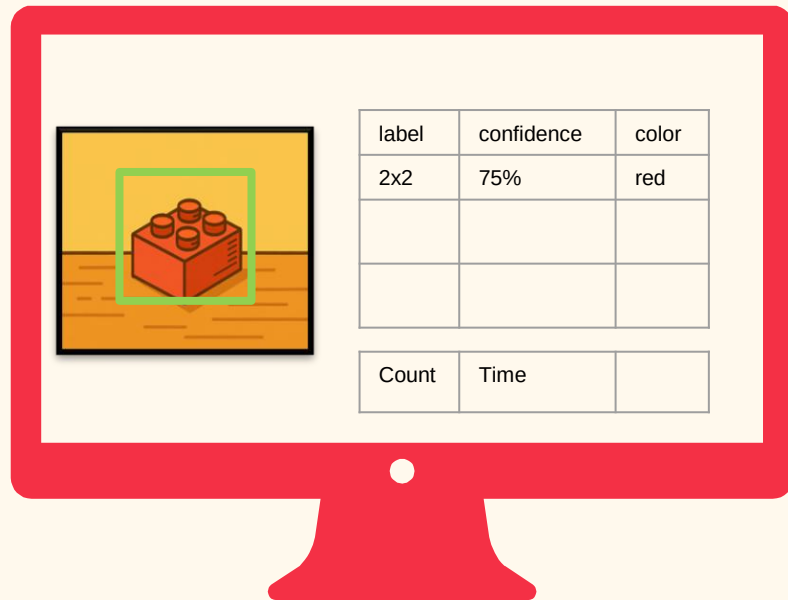
7. RESULT LOGGING AND DISPLAY

- Each result includes:
 - Detected part
 - Confidence
 - Matched colour
- Visuals (box + label) are drawn on the frame, and logs are updated.



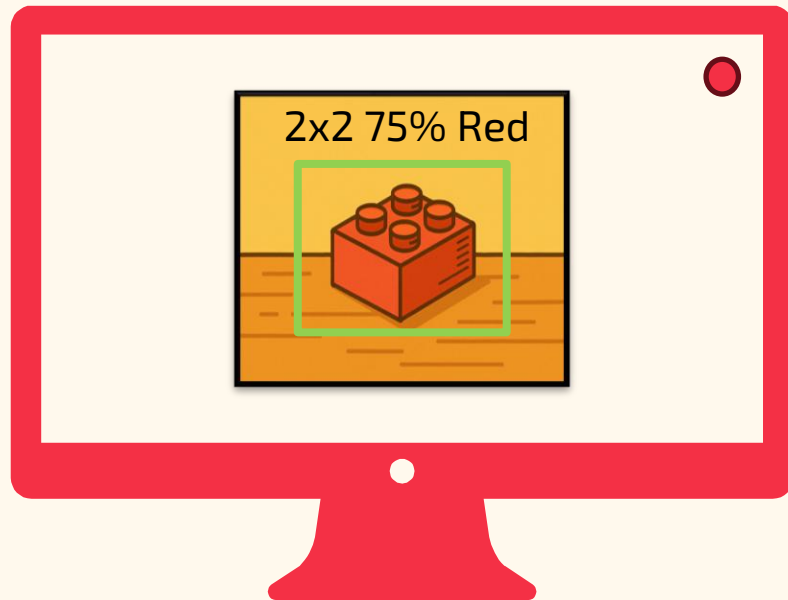
8. LOGGING & EXPORT

- Log each detection (label, confidence, color)
- Count parts and session time
- Export to Excel every 30s or manually

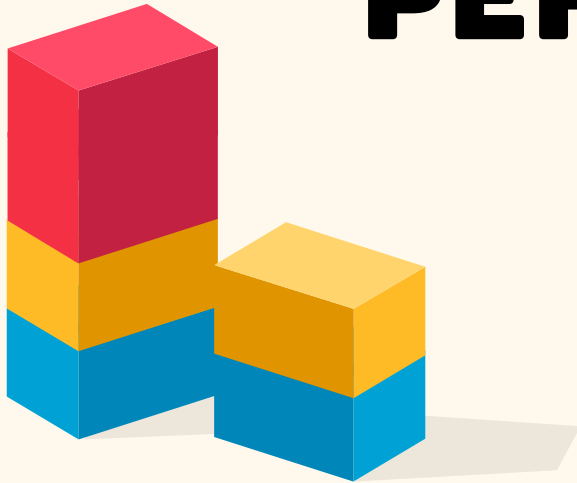


9. CONTINUOUS OPERATION

- Process repeats on each frame:
- Detection, classification, colour analysis, rendering
- GUI allows:
 - Switching camera sources
 - Classifying saved images
 - Saving session logs



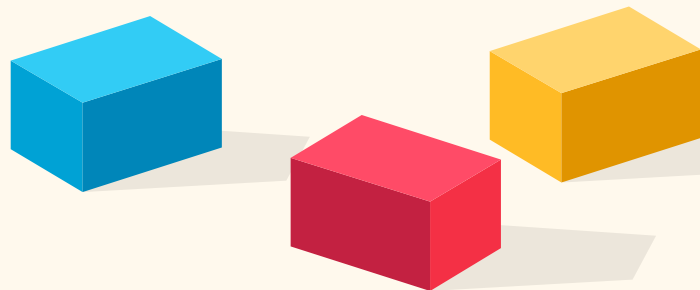
RESULTS & PERFORMANCE



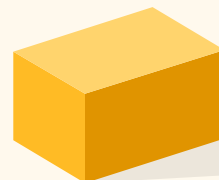
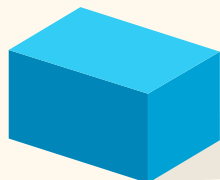
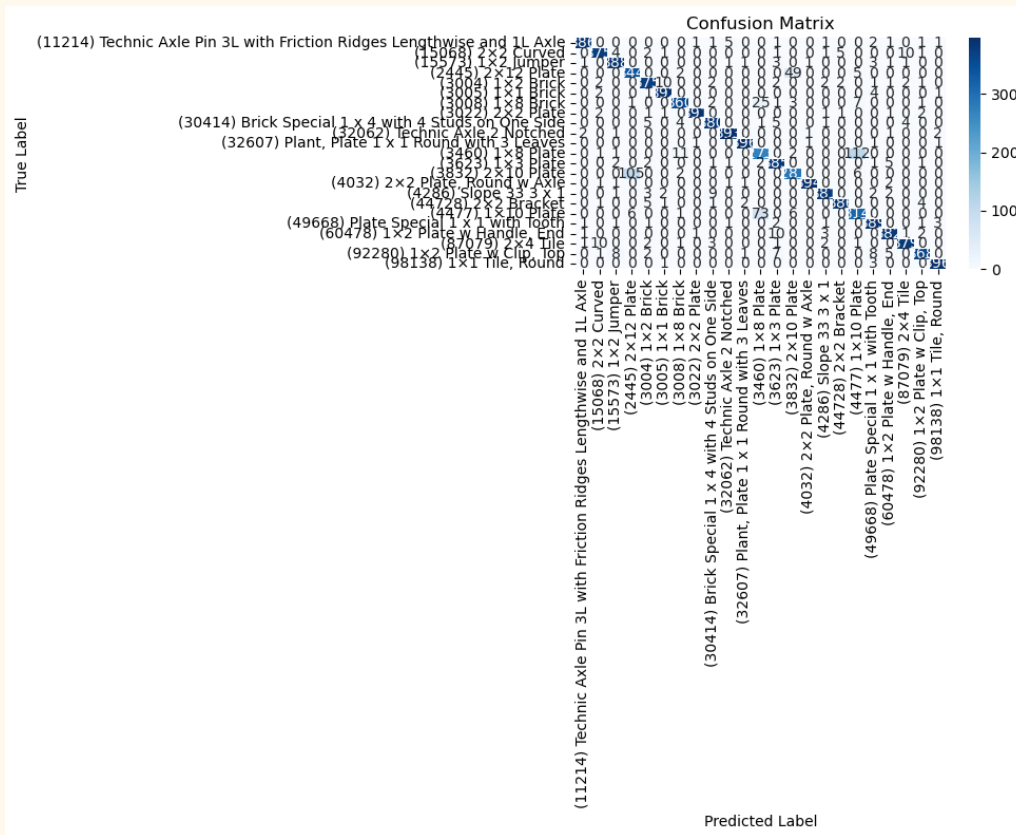
CNN23 RESULTS

ID	Part Name	Precision	Recall	F1 Score	Support
(11214)	Technic Axle Pin 3L	0.98	0.96	0.97	400
(15068)	2×2 Curved	0.95	0.94	0.94	400
(15573)	1×2 Jumper	0.95	0.97	0.96	400
(2445)	2×12 Plate	0.75	0.86	0.8	400
(3004)	1×2 Brick	0.94	0.94	0.94	400
(3005)	1×1 Brick	0.96	0.98	0.97	400
(3008)	1×8 Brick	0.94	0.9	0.92	400
(3022)	2×2 Plate	0.99	0.98	0.98	400
(30414)	Brick Special 1 x 4 with 4 Studs	0.95	0.95	0.95	400
(32062)	Technic Axle 2 Notched	0.98	0.98	0.98	400
(32607)	Plant, Plate 1 x 1 Round	0.99	0.99	0.99	400
(3460)	1×8 Plate	0.73	0.68	0.71	400
(3623)	1×3 Plate	0.92	0.96	0.94	400
(3832)	2×10 Plate	0.83	0.72	0.77	400
(4032)	2×2 Plate, Round w Axle	0.99	0.98	0.99	400
(4286)	Slope 33 3 x 1	0.97	0.95	0.96	400
(44728)	2×2 Bracket	0.98	0.96	0.97	400
(4477)	1×10 Plate	0.71	0.79	0.74	400
(49668)	Plate Special 1 x 1 with Tooth	0.94	0.97	0.96	400
(60478)	1×2 Plate w Handle, End	0.95	0.95	0.95	400
(87079)	2×4 Tile	0.95	0.95	0.95	400
(92280)	1×2 Plate w Clip, Top	0.96	0.92	0.94	400
(98138)	1×1 Tile, Round	0.98	0.99	0.99	400

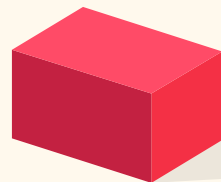
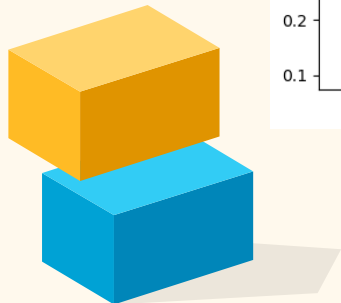
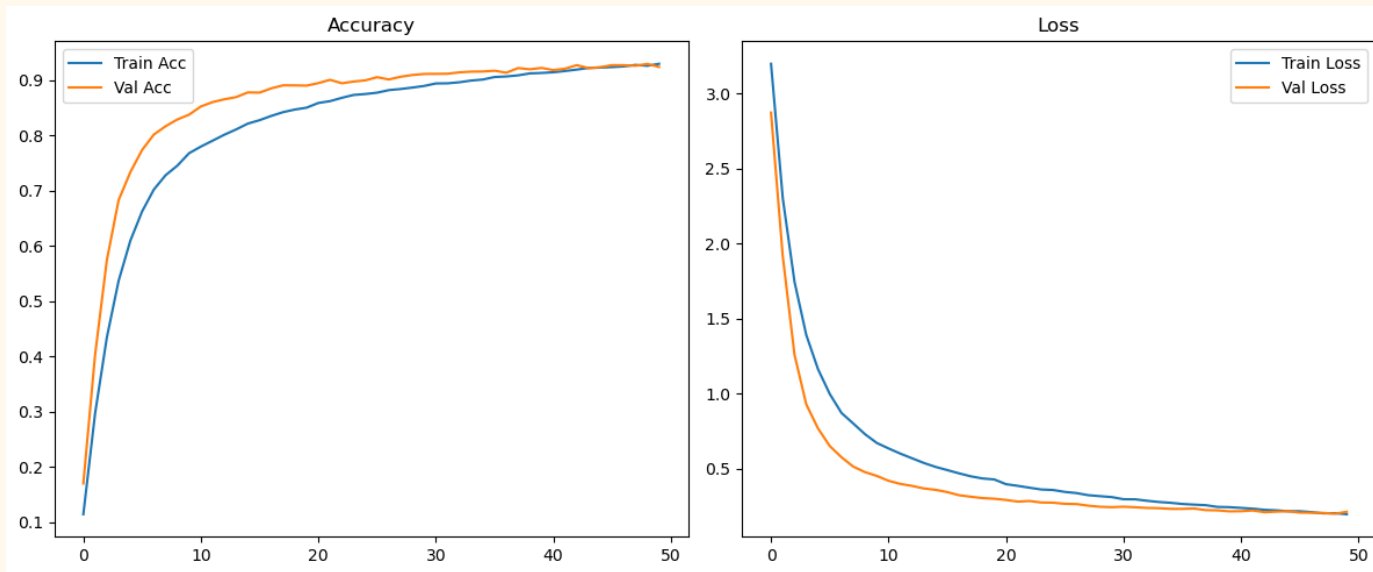
Overall accuracy	0.93
loss	0.1984
Validation accuracy	0.9234
Validation loss	0.2126
macro avg F1	0.93
weighted avg F1	0.93



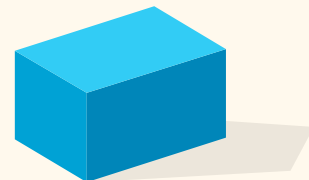
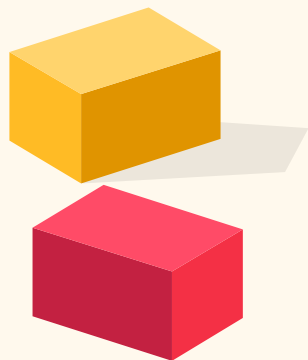
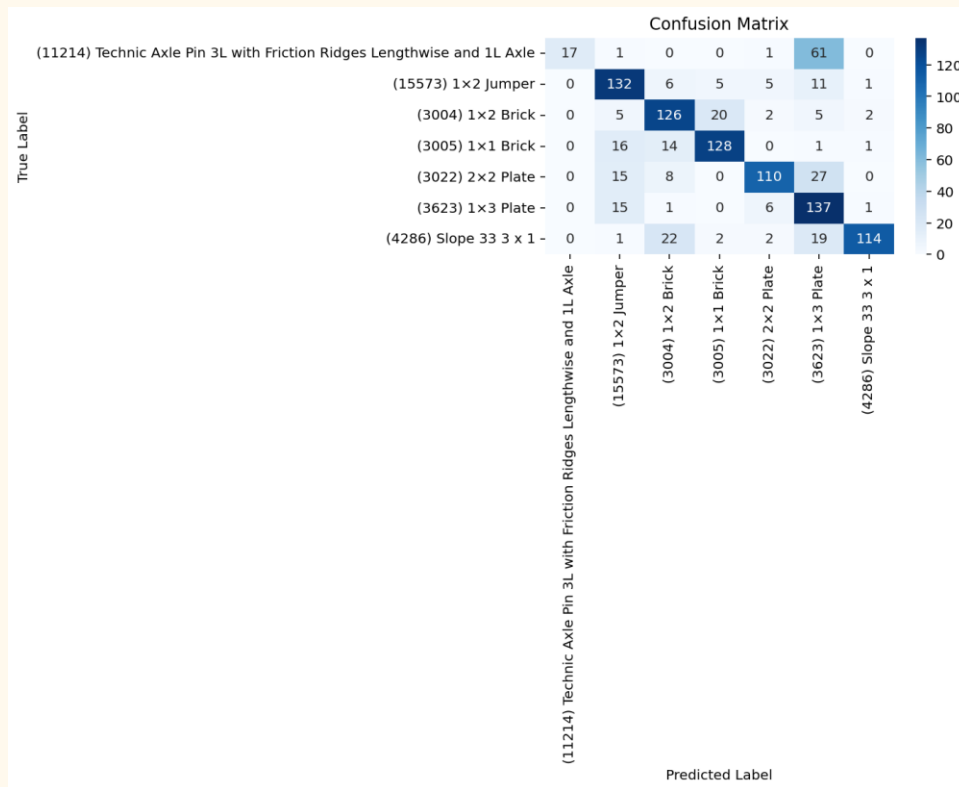
CNN23 RESULTS



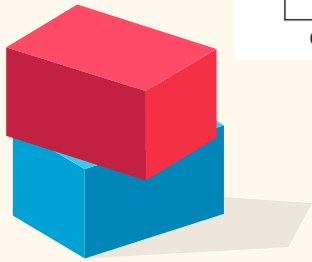
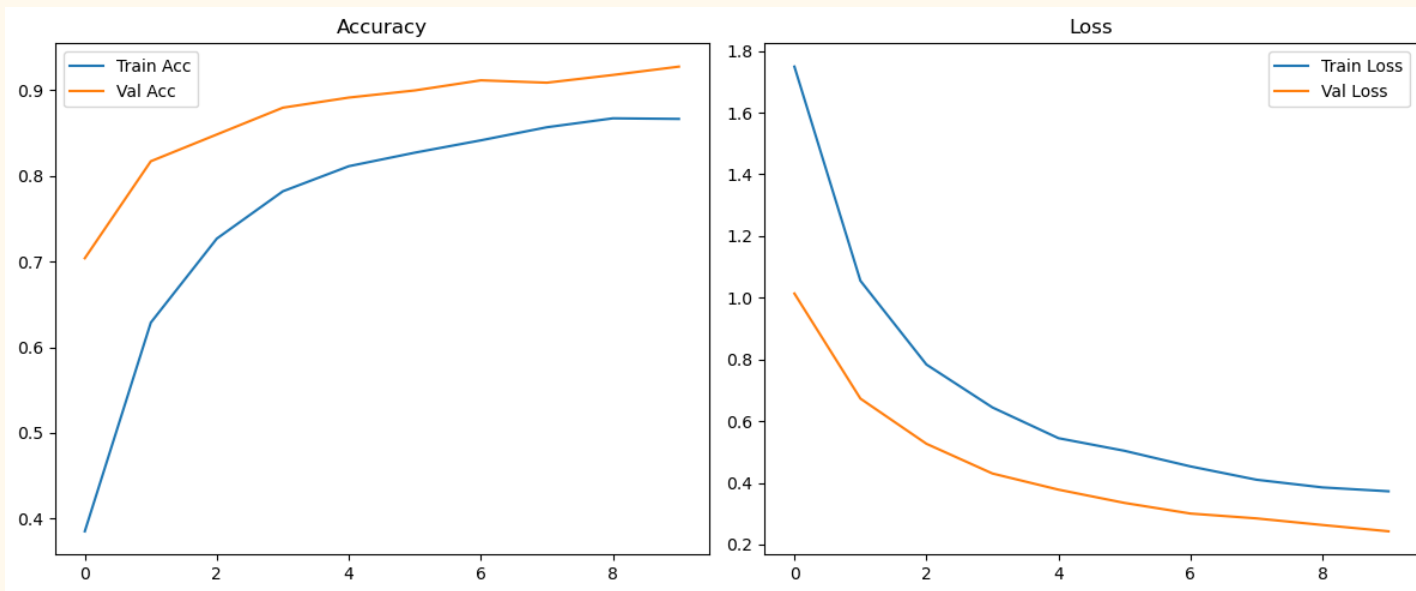
CNN23 RESULTS



CNN? RESULTS

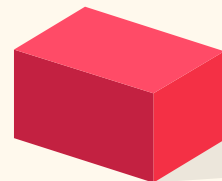
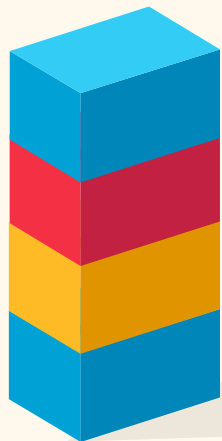
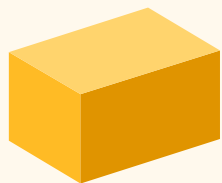


CNN7 RESULTS



YOLOV11 RESULTS

Class ID	Images	Instances	Box Precision	Recall	mAP@50	mAP@50-95
0	84	106	0.928	0.745	0.912	0.882
1	91	121	0.82	0.851	0.929	0.895
2	87	110	0.991	1	0.994	0.915
3	99	136	0.983	0.985	0.995	0.88
4	78	99	0.978	0.96	0.991	0.943
5	100	130	0.999	1	0.995	0.94
6	94	120	0.983	0.978	0.989	0.847
7	101	130	0.912	0.957	0.982	0.944
8	98	137	0.993	0.969	0.993	0.915
9	95	114	0.956	1	0.99	0.965
10	114	139	0.983	0.957	0.991	0.917
11	93	128	0.983	0.992	0.993	0.883
12	104	133	0.985	0.972	0.991	0.946
13	113	153	1	0.962	0.995	0.93



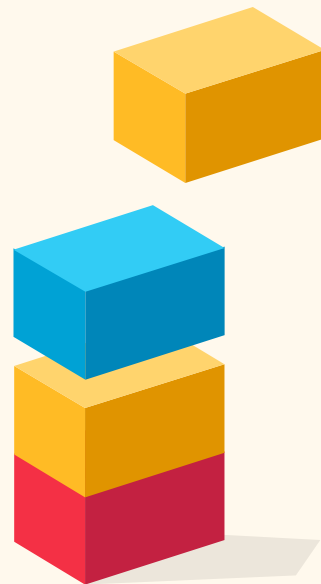
OVERALL MODEL RESULTS

Classification Performance

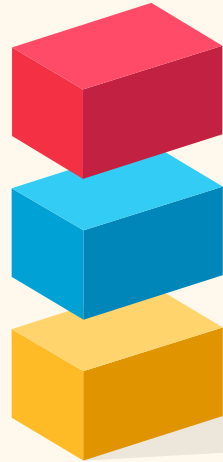
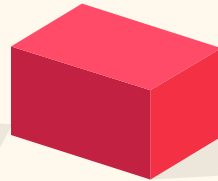
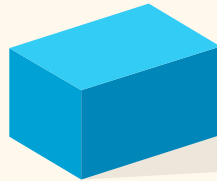
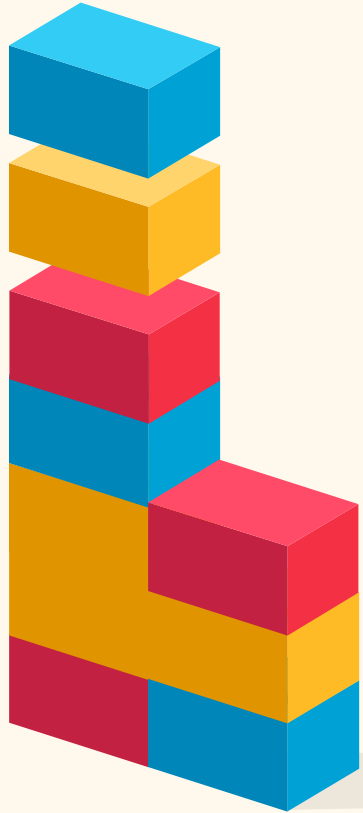
- CNN-23 Accuracy: 93.2% across 23 LEGO part classes
- CNN-7 Ensemble Boost: +3–5% accuracy on selected difficult parts
- YOLOv8 mAP@0.5: 89.5% for object detection
- Combined system accuracy (YOLO + CNN + fallback): ~92% in real-world use

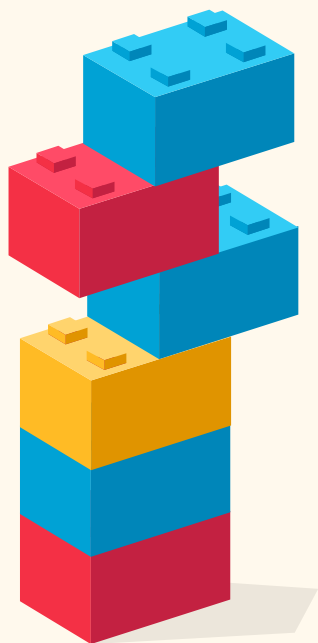
Robustness:

- Works reliably on small/tiny parts, even with partial occlusion
- Skip logic ensures certain parts are always classified by CNN
- Colour recognition accuracy: ~87% to closest LEGO shade

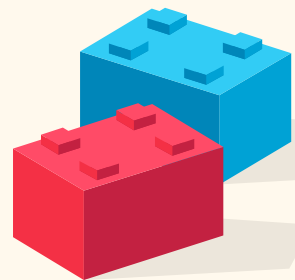


LIVE DEMO



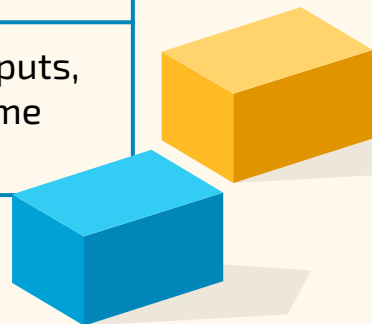
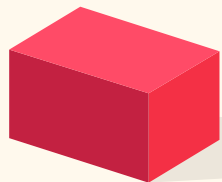


THE VIDEO IS INCLUDED IN THE ZIP FILE



WHAT WE OVERCAME

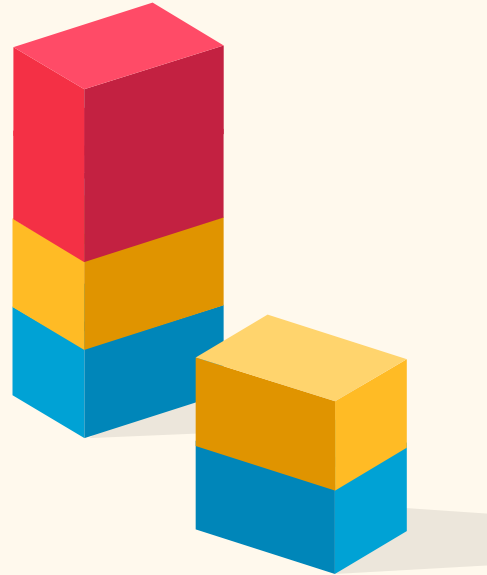
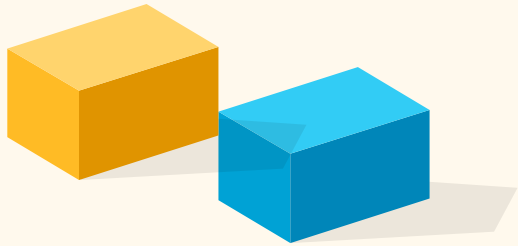
CHALLENGE	SOLUTION
YOLO struggles with some parts	Patch re-classification with MobileNetV2
Low confidence fallback	Centre crop classified if YOLO misses
Misclassification	Colour confirmation + ensemble thresholding
Real-time performance	Frame skipping, resized inputs, model optimization, frame smoothing

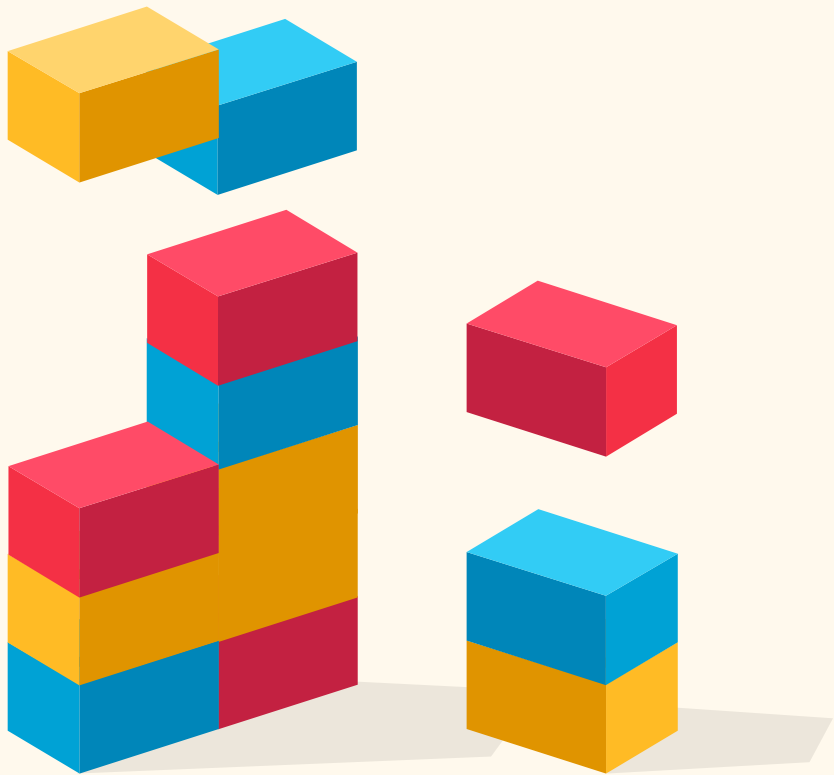


USE CASES

- LEGO sorting robots
- Inventory tracking
- Symbol learning
- Recycling and reuse systems
- Tiny parts identification

(Time intensive to classify manually)





THANKS!